

Iris Flower Classification

CodeAlpha DataScienceInternship-Task1|Step-by-step explanation and video script

What is this task?

We have measurements of 150 Iris flowers. Our goal is to build a machine learning model that looks at the 4 measurements of a flower and tells us its **species**: setosa, versicolor or virginica. This is a **classification** problem, because the answer is a category (a name), not a number.

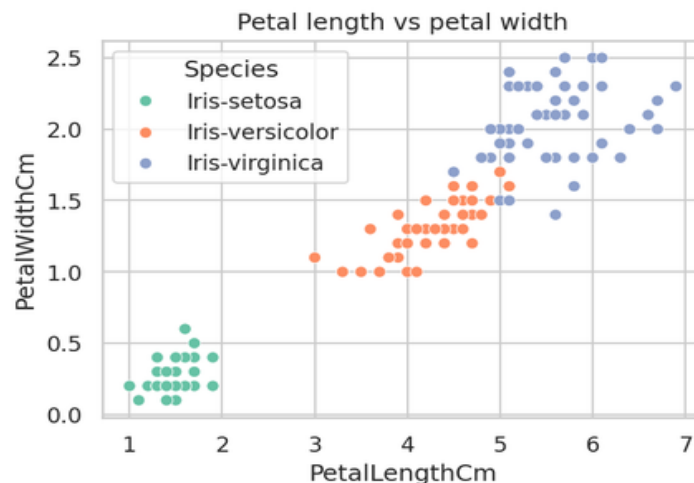
Part	Meaning	Columns
Input (X) - features	The 4 flower measurements	SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm
Output (y) - label	The species we want to predict	Species (setosa / versicolor / virginica)

Note: the species names are the **output**, not the input. The input is the measurements of flowers from these 3 species.

Step 1: Use flower measurements as input data

Each row of the dataset is one flower: 4 measurements plus its correct species. There are 150 flowers, 50 per species (balanced), and no missing values. We look at the data first (head, describe, class counts) to understand it before modelling.

```
df = pd.read_csv("Iris.csv")
X = df[["SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm"]] # input
y = df["Species"] # output
```



The chart shows that setosa (small petals) is clearly separate. Versicolor and virginica overlap a little, so mistakes can happen there.

Step 2: Train a model to classify the species

Training means showing the model many flowers with their correct species so it learns patterns. This is **supervised learning** because the data has correct answers.

2a. Split into train and test

We cannot test a model on the same data it learned from, like giving students the exam questions in advance. So we keep 80% (120 flowers) for training and 20% (30 flowers) for testing.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

random_state=42 gives the same split every run. **stratify=y** keeps equal species proportions in both parts.

2b. Train the models

We train 5 algorithms (Logistic Regression, KNN, Decision Tree, SVM, Random Forest) and compare them. **fit()** means learn from the training data.

```
model = SVC(kernel="rbf")
model.fit(X_train, y_train)
```

Step 3: Use Scikit-learn for data access and model building

Scikit-learn is a Python library with ready-made tools, so we do not write algorithms from scratch.

Job	Scikit-learn tool
Get Iris data	load_iris()
Split data	train_test_split
Scale numbers	StandardScaler
Models	SVC, KNeighborsClassifier, DecisionTreeClassifier, RandomForestClassifier, LogisticRegression
Check performance	accuracy_score, classification_report, confusion_matrix

Scaling: models like SVM and KNN work better when features are on a similar scale. We put StandardScaler inside a **pipeline** so it learns only from training data (this prevents data leakage).

Step 4: Evaluate accuracy and performance on test data

We give the 30 unseen test flowers to the model, let it predict, and compare with the real species.

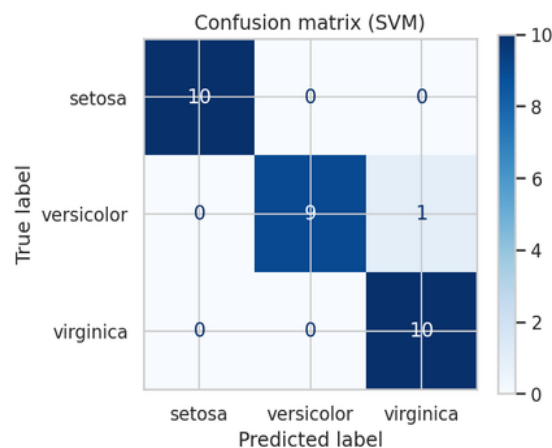
```
y_pred = model.predict(X_test)
print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Metric	Simple meaning
Accuracy	Correct predictions divided by total predictions
Precision	When the model says 'virginica', how often is it right?
Recall	Of all real virginica flowers, how many did the model find?
F1-score	One number combining precision and recall
Confusion matrix	Table showing which species were mixed up
Cross-validation	Test 5 times on different parts of the training data for a more reliable result

Our results

Model	Test accuracy	CV mean accuracy
SVM (RBF) - best	0.967	0.967
Logistic Regression	0.933	0.958
K-Nearest Neighbors	0.933	0.958
Decision Tree	0.933	0.950
Random Forest	0.900	0.950

The best model (SVM) got **29 out of 30** test flowers right (96.7%). Setosa: 10/10, virginica: 10/10, versicolor: 9/10 (one was predicted as virginica).



Step 5: Basic classification concepts

- Classification:** predicting a category (species name, spam / not spam).
- **Features (X) and label (y):** the inputs and the correct answer.
- **Train and test data:** learn on one part, check on another.
- Overfitting:** the model memorizes training data and fails on new data. The test set helps us catch it.
- Model comparison:** try several algorithms and choose the best using cross-validation.
- Metrics:** accuracy alone is not enough, so we also check precision, recall and the confusion matrix.
- Data leakage:** test data must never influence training.

Whole process: Load data > Explore > Split train/test > Train models > Test and compare > Pick best > Predict new flowers